GrammarCoach: AI Grammar Fixer & Learning Tool

Nov 1, 2026 · @nehal

Title Page

Project title	GrammarCoach: An Interactive AI Grammar Fixer and Learning Tool Using a T5 Transformer
Course	Professional Training (Intermediate level)
Course code	YN3012170034
Assignment	Mid-term Individual Project
Student name	ALHAMMADI NEHAL
Student ID	20243120040
Repository folder	`alh040` in xahiru/PT22026
Submission date	Week 12, 2026

Abstract

Millions of people write in English as a second language, and most automatic grammar checkers only return a corrected sentence without explaining what was wrong. This project presents **GrammarCoach**, an interactive web application that corrects English grammar with a Transformer model and turns every correction into a short, categorised grammar lesson. The system uses the pre-trained model `vennify/t5-base-grammar-correction`, a T5-base encoder-decoder (about 220 million parameters) fine-tuned on the JFLEG corpus of learner English. A rule-based feedback engine aligns the original and corrected text word by word, classifies each edit into one of 15 mistake categories (for example verb tense, articles and subject-verb agreement), and attaches an explanation. The application, built with Gradio, also offers a practice quiz on real learner sentences, scored against four human corrections, and a progress view that charts the learner's most frequent mistake types.

The model was evaluated on the 747-sentence JFLEG test set against a "no correction" baseline. GLEU rose from 0.787 to 0.911, similarity to the closest human correction rose from 91.2% to 97.0%, and the share of sentences exactly matching a human correction rose from 26.6% to 61.3%. The model improved 62.1% of sentences and made 7.4% worse. The results show that a compact pre-trained Transformer, combined with simple linguistic rules, can provide useful correction and learning feedback on an ordinary laptop CPU, while also revealing limits around over-correction, rule-based labelling and possible overlap between the model's training data and the test set.

Keywords: grammatical error correction, T5, Transformer, natural language processing, language learning, Gradio, Hugging Face

1. Introduction

1.1 Motivation

English is the main language of international study, science and business, and a large share of the people who write in it learned it as a second or foreign language. These writers make predictable mistakes: wrong verb tenses, missing or wrong articles (*a*, *an*, *the*), subject-verb disagreement, wrong prepositions and spelling errors. Teachers can correct such mistakes, but personal feedback is slow and limited by class size.

Automatic **grammatical error correction (GEC)** systems address this by rewriting a sentence into a corrected form. Modern GEC systems are based on neural networks, especially the Transformer architecture, and reach high accuracy. However, most tools return only the corrected text. Language-learning research shows that learners benefit most when feedback helps them notice *why* a form is wrong, not just *what* the right form is. A tool that silently fixes text may help a writer finish a document, but it does little to help them stop making the same mistake.

This project was motivated by a situation familiar to international students: writing assignments in English and wanting both a correction and an explanation, in one place, without paying for a commercial tool.

1.2 Problem Statement

The project addresses the following problem: *How can a pre-trained Transformer model for grammar correction be turned into an interactive application that not only corrects English text but also explains each mistake and helps the user practise?*

This involves three sub-problems:

1. **Correction:** produce a grammatical version of the user's text with a machine learning model.

2. **Explanation:** identify each individual change and describe it in learner-friendly terms.
3. **Practice and evaluation:** let the user practise on real learner errors, and measure objectively how well the model performs.

1.3 Objectives

1. Integrate a pre-trained sequence-to-sequence Transformer (T5) from the Hugging Face Model Hub to correct English sentences and paragraphs.
2. Design a feedback engine that detects every edit and classifies it into understandable mistake categories with a short lesson.
3. Build a user-friendly web interface in Python with Gradio, with tabs for correction, a practice quiz, progress tracking and model explanation.
4. Evaluate the model quantitatively on a standard benchmark (JFLEG) against a baseline, and discuss its strengths and limitations.
5. Follow good software practice: modular code, error handling, unit tests, documentation, and version control with Git and GitHub.

1.4 Contributions

- A working, reproducible application (**GrammarCoach**) that combines neural correction with rule-based, categorised explanations in 15 categories.
- A **learning mode** in which the user corrects real learner sentences and is scored against four human corrections, with hints for the remaining mistakes.
- A **progress tracker** that aggregates the user's mistake types and recommends what to study next.
- An **evaluation pipeline** that reports GLEU, similarity, exact match and the shares of improved and worsened sentences, with charts.

1.5 Report Structure

Section 2 reviews related work on grammatical error correction and corrective feedback. Section 3 describes the system design, the T5 model and the evaluation method. Section 4 covers the implementation. Section 5 presents and discusses the results. Section 6 concludes and proposes future work.

2. Related Work

Grammatical error correction has moved from hand-written rules, to statistical machine translation, to neural sequence-to-sequence models, and finally to large pre-trained Transformers such as T5, which this project uses.

2.1 Rule-Based Grammar Checking

The earliest grammar checkers used hand-written rules and patterns, for example *"a before a word starting with a vowel sound should be an"*. The open-source tool LanguageTool still follows this approach with thousands of rules. Rule-based systems are fast, transparent and easy to explain, which suits teaching. However, they cover only the errors their authors anticipated and handle complex or context-dependent mistakes poorly. GrammarCoach reuses the *strength* of rules (transparency) in its feedback engine, while relying on a neural model for the correction itself.

2.2 GEC as Machine Translation

A major shift came from treating GEC as translation from "incorrect English" into "correct English". Junczys-Dowmunt and Grundkiewicz (2016) showed that phrase-based statistical machine translation, tuned for the task, reached state-of-the-art results. Shared tasks such as CoNLL-2014 (Ng et al., 2014) and BEA-2019 (Bryant et al., 2019) provided common datasets and metrics and accelerated progress.

2.3 Neural and Transformer-Based GEC

Yuan and Briscoe (2016) applied neural encoder-decoder models to GEC, learning corrections directly from data. The Transformer (Vaswani et al., 2017) replaced recurrence with **self-attention**, letting each word attend to every other word in the sentence. Transformers quickly became the standard GEC architecture (Junczys-Dowmunt et al., 2018). An alternative line, GECToR (Omelianchuk et al., 2020), predicts an edit tag per token instead of rewriting the sentence, which is faster but needs a hand-designed tag set.

Pre-trained language models then raised accuracy further. **T5** (Raffel et al., 2020) frames every NLP task as text-to-text: a task prefix such as `grammar:` is added to the input and the model generates the output text. Rothe et al. (2021) showed that T5 models fine-tuned for GEC achieve strong results in several languages. The model used in this project, `vennify/t5-base-grammar-correction`, follows this recipe: a T5-base model fine-tuned on JFLEG.

2.4 Datasets and Evaluation Metrics

JFLEG (Napoles, Sakaguchi and Tetreault, 2017) contains sentences written by English learners, each corrected by four annotators. Unlike earlier corpora that focus on minimal grammatical edits, JFLEG rewards **fluency**, meaning corrections that sound natural to a native speaker. Because a sentence can often be corrected in several valid ways, metrics compare the output against multiple references. GLEU (Napoles et al., 2015) is the

standard JFLEG metric and is based on n-gram overlap, similar to BLEU in machine translation. This project uses the closely related sentence-level GLEU defined by Wu et al. (2016), together with simpler similarity and exact-match measures (Section 3.6).

ERRANT (Bryant, Felice and Briscoe, 2017) automatically extracts the edits between an original and a corrected sentence and classifies them into error types. GrammarCoach's feedback engine follows the same idea in a simplified, learner-oriented form.

2.5 Commercial and Educational Tools

Commercial products such as Grammarly combine neural models with explanations, but they are closed-source, partly paid, and give no insight into how they work. Open tools usually provide either correction (neural models) or explanation (rules), rarely both, and seldom include practice exercises.

2.6 Corrective Feedback in Language Learning

Second-language research has long debated the value of written grammar correction. Truscott (1996) argued that grammar correction is often ineffective, while studies such as Lyster and Ranta (1997) found that feedback which pushes learners to notice and repair their own errors leads to more learner "uptake". This supports GrammarCoach's design choices: explaining *why* each change was made, letting learners correct sentences themselves in the quiz, and showing which error types recur.

2.7 Summary and Gap

Approach	Corrects text	Explains mistakes	Practice mode	Open and runnable locally
Rule-based checkers (e.g. LanguageTool)	Partly	Yes	No	Yes
Neural GEC models (e.g. T5, GECToR)	Yes	No	No	Yes
Commercial tools (e.g. Grammarly)	Yes	Yes	Limited	No
GrammarCoach (this project)	**Yes**	**Yes**	**Yes**	**Yes**

The gap this project fills is a small, open and understandable application that pairs a neural corrector with transparent explanations and active practice.

3. System Design and Methodology

GrammarCoach separates the one machine learning component (the T5 model) from transparent, rule-based logic, so that every explanation the learner sees can be traced to a simple rule.

3.1 System Overview

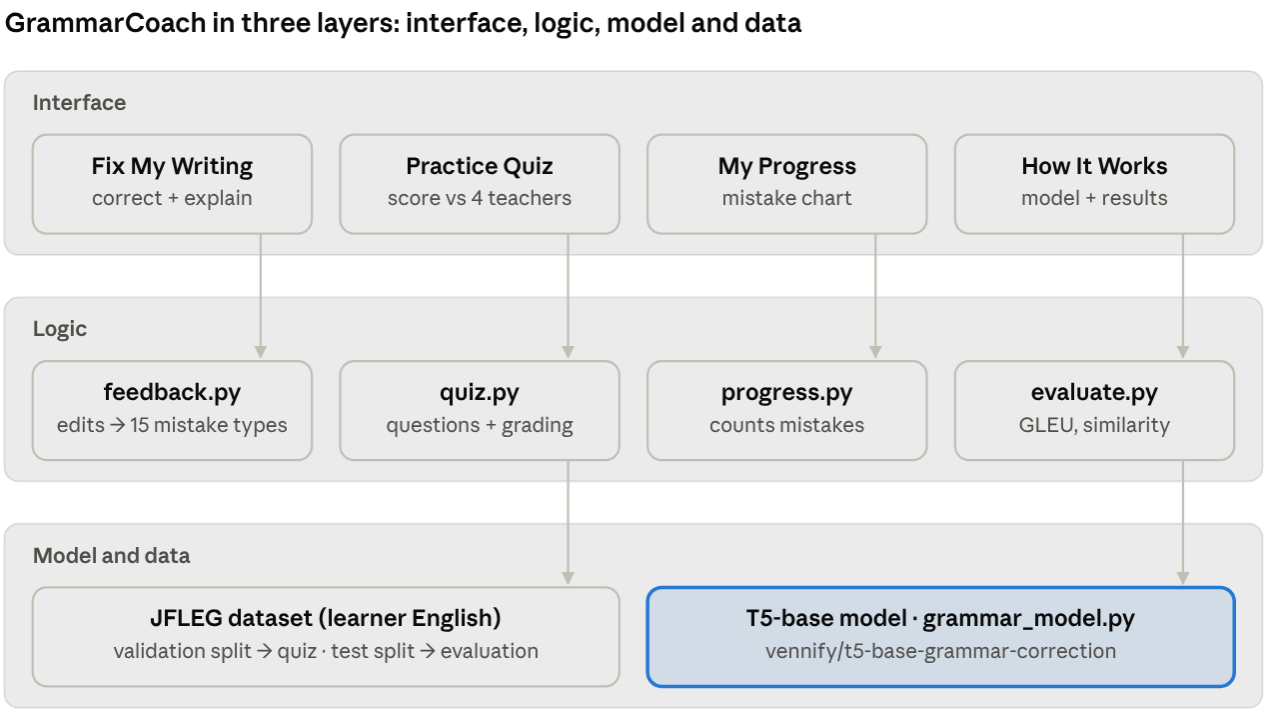


Figure 1 · GrammarCoach architecture: 4 tabs, 4 logic modules, 1 model, 1 dataset

Each tab in the interface calls one logic module, and the Fix My Writing tab and the quiz's "Show AI's correction" button also call the T5 model directly. The evaluation script uses both the model and the JFLEG test split.

A typical request in the **Fix My Writing** tab follows these steps:

1. The user types or pastes English text and clicks *Check my grammar*.
2. The text is validated (not empty, at most 3,000 characters) and split into sentences.
3. The T5 model corrects each sentence (in batches of 8, using beam search).
4. The feedback engine compares the original and corrected text, lists every edit, labels its mistake type and attaches a mini-lesson.
5. The interface shows a correctness score, the changes in colour (red = removed, green = added), the corrected text and the lessons. The mistake counts are added to the session's progress.

3.2 The Machine Learning Model: T5

Property	Value
Model	`vennify/t5-base-grammar-correction` (Hugging Face Model Hub)
Architecture	T5-base encoder-decoder Transformer (Raffel et al., 2020)
Size	about 220 million parameters (12 encoder + 12 decoder layers, hidden size 768, 12 attention heads)
Tokenizer	SentencePiece, vocabulary of 32,000 sub-word pieces
Pre-training	C4 web-text corpus, "span corruption" objective (fill in masked spans of text)
Fine-tuning	JFLEG learner sentences paired with human corrections, input prefix `grammar:`
Licence	CC BY-NC-SA 4.0 (non-commercial use)

Why T5 was chosen. T5 treats grammar correction as text-to-text generation, which naturally handles any kind of edit: insertions, deletions, replacements and reordering. The base size runs on an ordinary laptop CPU in about 2 seconds per sentence. The model is openly available and was fine-tuned on learner English, which matches the target users.

How the model processes a sentence.

1. **Tokenization.** The input `grammar: Yesterday i go to school.` is split into sub-word pieces and converted into numbers (token IDs). Sub-words let the model handle misspelled or rare words, because any word can be built from smaller pieces.
2. **Encoder.** Twelve Transformer layers read the whole sentence at once. In each layer, **self-attention** lets every token gather information from every other token. This is how the model learns that *go* conflicts with *Yesterday*: the attention weights link the two words, and the representation of *go* now carries the information that a past-tense context surrounds it.
3. **Decoder.** Twelve more layers generate the corrected sentence one token at a time. Each step uses masked self-attention over the tokens already written, plus **cross-attention** to the encoder's representation of the original sentence. A final softmax layer gives a probability for each of the 32,000 vocabulary pieces.

4. **Beam search.** Instead of always picking the single most likely next token (greedy decoding), the model keeps the 5 most probable partial sentences at each step and returns the best complete one. This avoids early mistakes that greedy decoding cannot undo.

The core operation in each attention layer is scaled dot-product attention (Vaswani et al., 2017), where *Q*, *K* and *V* are the query, key and value matrices computed from the token representations and *d* is the key dimension:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^{T} / \sqrt{d} \right) \cdot V$$

During fine-tuning, the model was trained to minimise the cross-entropy loss between its predicted tokens and the human-corrected sentence *y*, given the learner sentence *x*:

$\text{Loss} = -\sum_{t} \log P(y_t \mid y_1 \ldots y_{t-1}, x)$, summed over every token t of the corrected sentence.

This project uses the fine-tuned model as published and does not train it further. The project's own contribution is the system around it: the feedback engine, the learning features and the evaluation.

3.3 The Feedback Engine

The model outputs only a corrected sentence. The feedback engine (`feedback.py`) turns that output into learning feedback in four steps.

1. **Tokenize.** Both texts are split into words and punctuation marks with a regular expression (for example `I don't know.` becomes `I` , `don't` , `know` , `.`).
2. **Align.** Python's `difflib.SequenceMatcher` aligns the two token lists (compared in lower case) and returns the blocks that were kept, replaced, inserted or deleted. Large blocks are split into smaller edits, so that "buyed a" → "bought an" becomes two edits with two lessons, and punctuation changes are separated from word changes. A word removed in one place and added in another is merged into a single *word order* edit.
3. **Classify.** Each edit is labelled by an ordered set of linguistic rules, checked from most to least specific:

Mistake category	Rule used (simplified)	Example
Punctuation	only punctuation changed, or only an apostrophe	dont → don't
Capitalization	same word, different capital letters	i → I
Word order	same words in a different order	like really → really like
Articles (a / an / the)	only articles added, removed or changed	a apple → an apple

Mistake category	Rule used (simplified)	Example
Commonly confused words	both words in a confusable group	their → they're
Subject-verb agreement	known agreement pairs, or -s added after a subject pronoun	they is → they are
Verb tense	known tense pairs, -ed endings, irregular verb list	go → went, buyed → bought
Singular / plural	noun gains or loses -s / -es / -ies	book → books
Prepositions	both words in the preposition list	in Monday → on Monday
Pronouns	both words in the pronoun list	him → he
Word form	same word root, different ending	interesting → interested
Spelling	character similarity at least 75%	tomorow → tomorrow
Missing word	words only added	added "the"
Extra word	words only removed	removed "very"
Word choice	anything else	make → do

4. **Explain and display.** Each category has a short lesson with an example (for example, for verb tense: *use the past tense for finished actions: "Yesterday I went home"*). The engine also produces coloured HTML and a **correctness score**, the percentage of the original words that the AI kept unchanged:

Score = 100 × matched tokens / max(tokens in original, tokens in corrected)

Rules were chosen for this step, instead of a second model, because they are transparent: a learner (or examiner) can see exactly why a label was given. The cost is that labels are a best guess for unusual edits (Section 5.4).

3.4 Learning Features

Practice Quiz. Questions come from the JFLEG **validation** split. Only sentences of 6 to 25 words whose four human corrections all differ from the original (so the sentence really contains a mistake) are kept, which leaves 436 practice sentences. The learner edits the sentence and the answer is compared with each human correction. The score (0 to 100) is

the word-level similarity to the closest one. The edits still needed to reach that correction are shown as hints with their lessons. If JFLEG cannot be downloaded, 12 built-in practice sentences written for the project are used instead.

My Progress. Every mistake found in the learner's own writing, and every mistake left in a quiz answer, is counted by category for the current browser session. The tab shows a bar chart of the categories, the average quiz score, and the three most frequent categories with their lessons as study recommendations. Nothing is saved to disk, which keeps the tool private.

3.5 Dataset

JFLEG (Napoles et al., 2017) is loaded from the Hugging Face Hub as `jhu-clsp/jfleg`. It contains sentences written by English learners, each with four corrections by human annotators: 755 rows in the validation split and 748 in the test split. The original text is tokenized with spaces before punctuation ("energy ."), so the project removes these spaces before use. Using the **validation split for the quiz** and the **test split for evaluation** ensures that the evaluation sentences are never shown in the quiz.

3.6 Evaluation Method

The model is evaluated on the JFLEG test split (747 sentences after removing one row without usable corrections). As a **baseline**, the original learner sentence is scored without any correction. A useful corrector must score clearly higher than this "do nothing" baseline. Five measures are reported:

- **GLEU** (Wu et al., 2016): the overlap of 1- to 4-word sequences (n-grams) between the output and the best-matching human correction. For a sentence, with *m* matching n-grams, GLEU is defined below; the corpus score sums matches and totals over all sentences. Note that the official JFLEG metric is a related GLEU variant (Napoles et al., 2015) that also penalises copying mistakes from the source, so the numbers here are not directly comparable with published leaderboards.
- **Similarity**: the word-level `difflib` similarity ratio (0 to 1) to the closest human correction, averaged over sentences.
- **Exact match**: the share of sentences identical to one of the four human corrections (ignoring capitalisation and spacing).
- **Changed, improved and worse rates**: the share of sentences the AI modified, and the share whose similarity to the human corrections went up or down after correction.

GLEU = min(m / n-grams in the output , m / n-grams in the reference)

4. Implementation

GrammarCoach is about 1,400 lines of Python in seven modules plus a test file, runs entirely on a laptop CPU, and is started with a single command, `python app.py`.

4.1 Tools and Environment

Component	Tool and version	Purpose
Language	Python 3.11	all code
Web interface	Gradio 5.50	tabs, text boxes, buttons, charts in the browser
Model loading	Hugging Face Transformers 4.57	downloads and runs the T5 model
Deep learning	PyTorch 2.15 (CPU)	tensor computation for the model
Tokenizer	SentencePiece 0.2	T5 sub-word tokenizer
Data	Hugging Face Datasets 5.0	downloads JFLEG
Charts	Matplotlib 3.11	progress and evaluation charts
Testing	pytest 9.2	unit tests
Version control	Git 2.56, GitHub	commits, fork and pull request
Editor and OS	Visual Studio Code on Windows	development

4.2 Project Structure

File	Responsibility
`app.py`	Gradio interface with four tabs; connects buttons to functions; error messages
`grammar_model.py`	loads the T5 model once; splits text into sentences; batched beam-search correction
`feedback.py`	tokenization, alignment, the 15 mistake categories, lessons, coloured diff, correctness score

File	Responsibility
`quiz.py`	loads and filters JFLEG validation sentences (with a local cache and built-in fallback); grades answers
`progress.py`	counts mistakes per session; summary text and bar chart
`metrics.py`	GLEU, corpus GLEU and exact match
`evaluate.py`	runs the model on the JFLEG test split; writes metrics, charts and an examples file to `results/`
`tests/test_core.py`	13 unit tests for the parts that do not need the model
`requirements.txt`, `.gitignore`, `README.md`	dependencies, excluded files, documentation

The code is **modular**: the interface (`app.py`) contains no grammar logic, and the logic modules contain no interface code. This made it possible to test the feedback engine, quiz grading and metrics without loading the 900 MB model.

4.3 Key Code

Correcting text with the model (`grammar_model.py`). The model and tokenizer are loaded once and kept in memory. Each sentence receives the `grammar:` prefix, and sentences are processed in batches:

```
batch = ["grammar: " + s for s in sentences[start:start + batch_size]]
inputs = tokenizer(batch, return_tensors="pt", padding=True,
                   truncation=True, max_length=128).to(device)
with torch.no_grad():
    outputs = model.generate(**inputs, num_beams=5,
                             max_new_tokens=128, early_stopping=True)
results.extend(tokenizer.batch_decode(outputs, skip_special_tokens=True))
```

`torch.no_grad()` turns off gradient tracking, which is only needed for training, saving memory and time.

Classifying an edit (`feedback.py` , simplified). Rules are checked from the most specific to the most general, so that, for example, *go → went* is labelled as verb tense rather than spelling:

```
if all(is_punct(w) for w in words):
    return "Punctuation"
if old == new:                      # same words, different capitals
    return "Capitalization"
if all(w in ARTICLES for w in words):
    return "Articles (a / an / the)"
if frozenset((a, b)) in AGREEMENT_PAIRS:
    return "Subject-verb agreement"
if frozenset((a, b)) in TENSE_PAIRS or is_regular_tense_change(a, b):
    return "Verb tense"
...
if similarity(a, b) >= 0.75:
    return "Spelling"
return "Word choice"
```

Connecting the interface (`app.py`). Each Gradio button is linked to a Python function, its inputs and its outputs. A `gr.State` object holds the learner's progress for the current browser session:

```
fix_btn.click(fix_writing,
              inputs=[text_in, progress_state],
              outputs=[corrected_out, diff_out, score_out, lessons_out,
progress_state])
```

4.4 User Interface

The interface has four tabs (Figure 1, top layer):

- **Fix My Writing.** A text box with five example sentences, a *Check my grammar* button, a score panel (percentage of words already correct, number of fixes), the coloured changes, the corrected text with a copy button, and one card per fix showing its category tag, the change and the lesson.
- **Practice Quiz.** A learner sentence, an answer box pre-filled with that sentence, and buttons to check the answer, show the AI's correction, or move to the next sentence. Feedback includes the score, the remaining differences in colour, hints with lessons, and the teachers' corrections.
- **My Progress.** A summary (texts checked, quiz questions answered, average score), the top three mistake types with lessons, a bar chart, and a reset button.
- **How It Works.** A plain-language explanation of T5, the feedback engine and the limitations, plus the evaluation table and charts produced by `evaluate.py`.

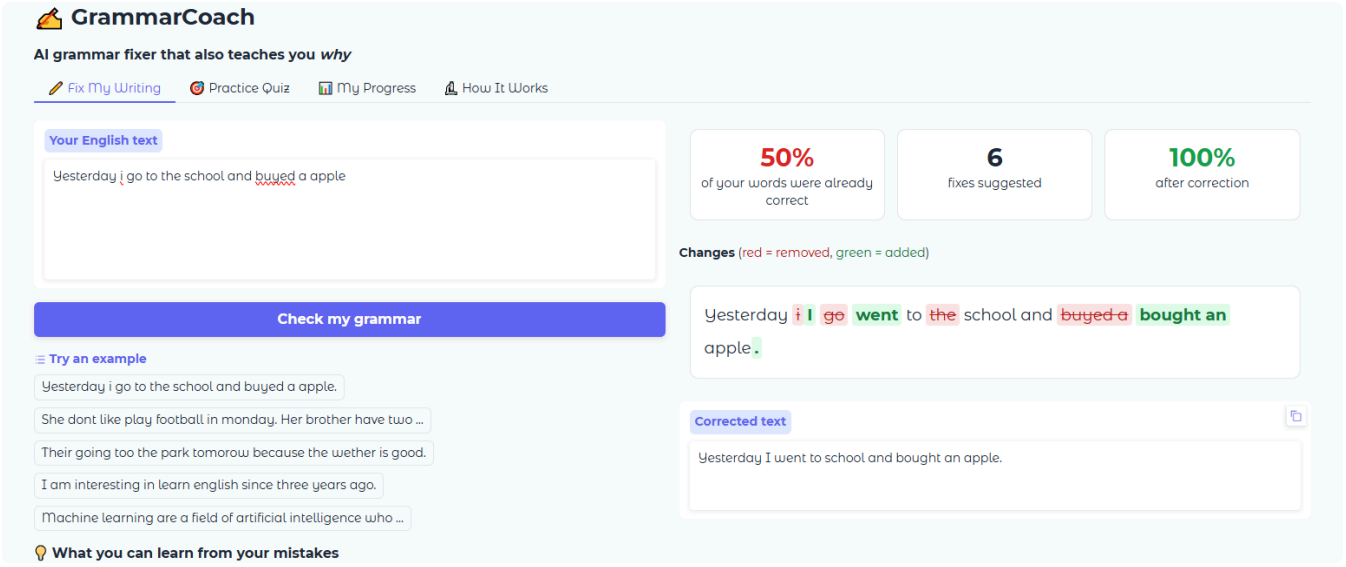


Figure 2. Fix My Writing: the model corrected "Yesterday i go to the school and buyed a apple" to "Yesterday I went to school and bought an apple." (6 fixes, 50% of words already correct).

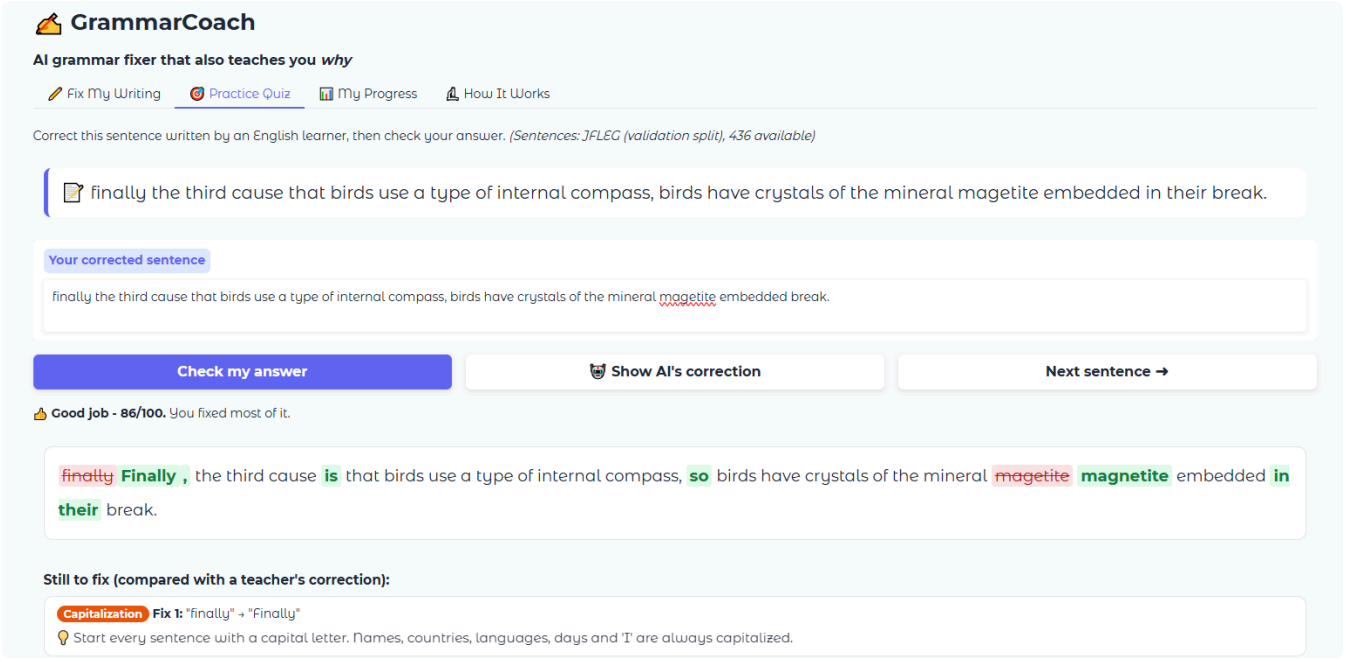


Figure 3. Practice Quiz: a real JFLEG learner sentence (436 available). The learner's answer scored 86/100; the remaining differences from the closest teacher correction are shown in colour, with a lesson for each.

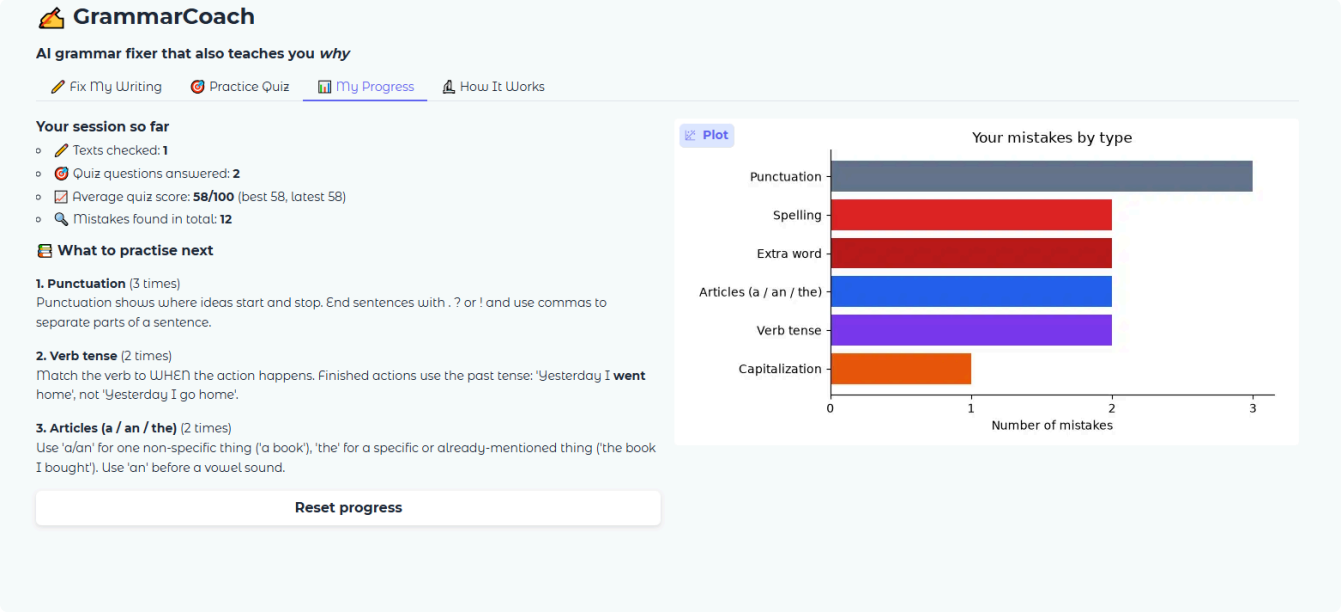


Figure 4. My Progress: 12 mistakes counted across one text and two quiz answers, with the top three categories as study recommendations.

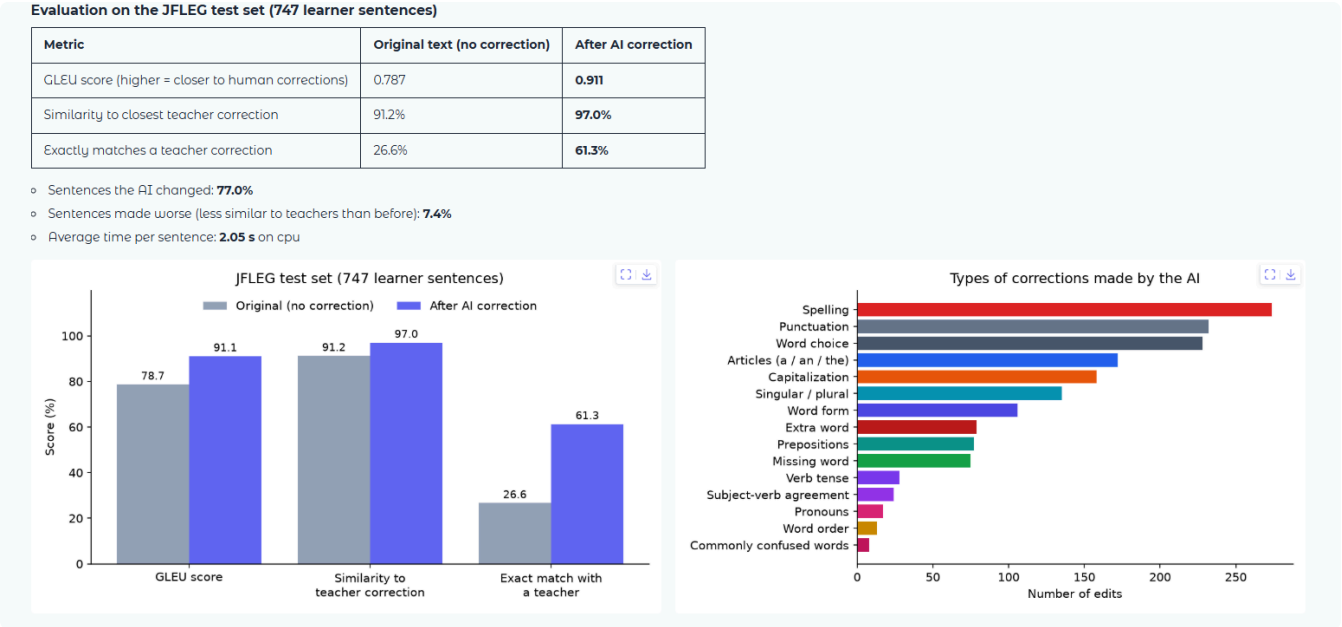


Figure 5. How It Works: the evaluation results produced by evaluate.py, with the score comparison and the types of corrections made by the AI on the JFLEG test set.

4.5 Error Handling and Problems Solved

Situation	Handling
Empty input	a clear message asks the user to type some text; the model is not called
Text longer than 3,000 characters	rejected with a message stating the limit, protecting the app from very long runs

Situation	Handling
Model cannot load (no internet on first run)	the app still starts and explains that the first run needs a connection
JFLEG cannot be downloaded	the quiz uses 12 built-in practice sentences; once downloaded, questions are cached locally
Hugging Face blocked in mainland China	the README explains how to use a VPN or the mirror `HF_ENDPOINT=https://hf-mirror.com`
Gradio failed to start with error 502	a VPN proxy intercepted requests to `127.0.0.1`; fixed by excluding local addresses from the proxy (`NO_PROXY`) before Gradio starts
A second 900 MB copy of the model was downloaded	Transformers tried to fetch a converted `safetensors` file; fixed by loading with `use_safetensors=False`

The last two problems appeared only when the application was first run on the target Windows laptop, which shows the value of testing in the real user environment.

4.6 Testing

Thirteen **unit tests** (`python -m pytest tests/`) check the parts that do not need the model: tokenization, each main mistake category (verb tense, agreement, articles, plurals, capitalization, spelling, confused words, word order), the coloured diff, GLEU and exact match, JFLEG detokenization, quiz grading for perfect and unchanged answers, and progress counting. All 13 pass. In addition, the full application and evaluation pipeline were tested end to end with a stand-in model before the real model was run, and then manually in the browser with the real model.

4.7 Version Control with Git and GitHub

The class repository `xahiru/PT22026` was forked to the student's account, cloned locally, and a branch `alh040-grammarcoach` was created. The project was added to the assigned folder `alh040` in eight commits, each covering one logical part:

1. Add project setup: requirements and gitignore
2. Add feedback engine: mistake categories, lessons and coloured diff
3. Add T5 grammar correction model wrapper
4. Add practice quiz and progress tracking

5. Add Gradio interface with four tabs
6. Add evaluation script, GLEU metric and unit tests
7. Add evaluation results on JFLEG test set
8. Add README with setup guide, model documentation and results

The `.gitignore` file keeps the virtual environment, downloaded data and model weights out of the repository; the model and dataset are downloaded automatically instead. The branch was pushed and submitted as pull request #2 to the main branch for instructor review; no files of other students were modified.

5. Results and Discussion

The T5 model outperformed the "no correction" baseline on every measure, and its output exactly matched a human teacher's correction for 61.3% of the test sentences, up from 26.6%.

5.1 Quantitative Results

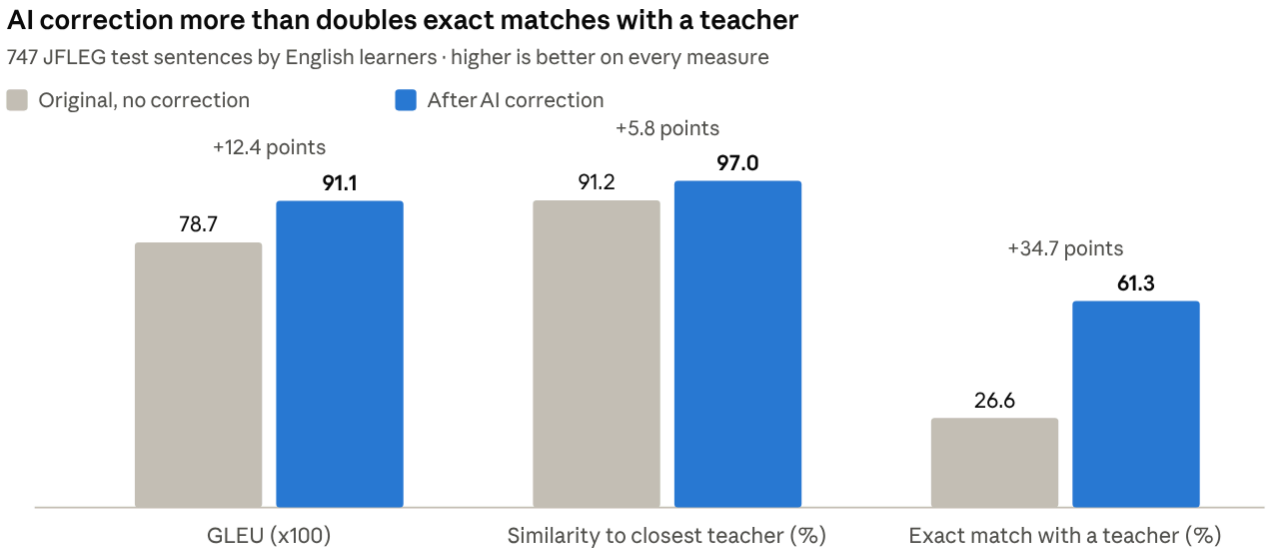


Figure 6 · results/metrics.json from evaluate.py · JFLEG test split, 747 sentences, laptop CPU

The model changed 77.0% of the sentences. Measured by similarity to the closest human correction, it **improved 62.1%** of sentences, left 30.5% at the same quality (including the 23.0% it did not change), and **made 7.4% worse**. For every sentence made worse, more than eight were improved.

Correction took **2.05 seconds per sentence** on average on the laptop CPU. The right-hand chart in Figure 5 shows the types of corrections the AI made across the test set, as labelled by the feedback engine. **Spelling** was the most frequent category (about 270 edits), followed by **punctuation** and **word choice** (about 230 each), **articles** (about 170)

and **capitalization** (about 160). Narrow grammar categories such as verb tense (about 30) and subject-verb agreement (about 25) were much rarer. Counts are read from the chart and are approximate.

5.2 Analysis

The baseline is already high, and that is expected. Without any correction, the learner sentences already score 0.787 GLEU and 91.2% similarity. Most words in a learner sentence are correct, and a typical correction changes only one to three words. The gains should therefore be read as a share of the remaining gap: the model closes **58%** of the GLEU gap to a perfect score ((0.911 − 0.787) / (1 − 0.787)) and **66%** of the similarity gap ((97.0 − 91.2) / (100 − 91.2)).

Exact match is the strictest measure. A sentence counts only if it is identical to one of four human corrections, word for word. Reaching 61.3% means that in most cases the model makes the same choices a human teacher would, not just grammatical ones. The 34.7-point increase is the clearest evidence that the model is useful.

Fluency, not only grammar. JFLEG references often rephrase awkward wording, not just fix grammar. Because the model was fine-tuned in this style, it also makes many fluency edits, which is why *word choice* is the third most frequent category. The high number of spelling and capitalization edits reflects the learner writing in JFLEG, which was produced under time pressure. The low counts for verb tense and agreement are partly a labelling effect: when such a fix comes together with other changes, the rules may file it under *word choice* instead.

5.3 Example of the Feedback Engine

The table below shows how the feedback engine explains one correction. The input is the first example sentence in the app, *"Yesterday i go to the school and buyed a apple."*, with the correction *"Yesterday I went to school and bought an apple."* Each row is one card shown to the learner.

Fix	Change	Category assigned	Lesson shown (shortened)
1	i → I	Capitalization	"I" is always capitalized
2	go → went	Verb tense	finished actions use the past tense
3	removed "the"	Articles (a / an / the)	"the" is for a specific thing
4	buyed → bought	Verb tense	*finished actions use the past tense; buy is irregular*

Fix	Change	Category assigned	Lesson shown (shortened)
5	a → an	Articles (a / an / the)	use "an" before a vowel sound

The T5 model produced exactly this correction when the sentence was entered in the running app (Figure 2), and all five edits were labelled correctly. The correctness score for this sentence was 55%, meaning just over half of the original words could stay unchanged.

The full list of 747 test sentences, with the AI correction, the closest teacher correction and the before and after scores, is available in `results/examples.csv` in the project repository.

5.4 Limitations

1. **Over-correction.** 7.4% of sentences became less similar to the human corrections. Sometimes the model rewrites a sentence that was acceptable, or chooses a valid correction that differs from all four references. Users should treat suggestions as advice, not as certain truth.
2. **Possible overlap between training and test data.** The model card states that the model was fine-tuned on JFLEG but does not say which splits were used. If the test split was included in training, the results above are optimistic, and performance on completely new writing would be lower. A fairer test would use a separate corpus, such as the BEA-2019 learner data.
3. **Rule-based labels are a best guess.** The 15 categories come from word lists and patterns, not from understanding. For example, *arts → art* is labelled *singular / plural* even when the real reason is word choice. Unusual edits fall back to *word choice*, which explains little.
4. **Sentence-by-sentence processing.** Each sentence is corrected separately, so errors that depend on the previous sentence (for example pronoun reference or tense consistency across a paragraph) can be missed.
5. **Input length and language.** Sentences longer than 128 sub-word tokens are truncated, and the model handles English only.
6. **Metric choice.** The GLEU used here (Wu et al., 2016) differs from the official JFLEG GLEU, so the numbers cannot be compared directly with published results. Quiz scores also reward similarity to only four references, so a different but correct answer may receive a lower score.
7. **Licence and speed.** The model's licence allows non-commercial use only, and correction on a CPU took 2.05 seconds per sentence on average, which is fine for learners but slow for very long documents.

5.5 Achievement of Objectives

Objective (Section 1.3)	Result
1. Integrate a pre-trained T5 model	Achieved: model loaded from the Hugging Face Hub and corrects sentences and paragraphs
2. Explain each edit	Achieved: 15 categories with lessons; correct labels on the tested learner errors
3. User-friendly interface	Achieved: four-tab Gradio app with examples, quiz and progress chart
4. Quantitative evaluation	Achieved: clear improvement over the baseline on all five measures
5. Good software practice	Achieved: modular code, error handling, 13 passing tests, README, eight commits and a pull request

6. Conclusion and Future Work

6.1 Conclusion

This project built **GrammarCoach**, an interactive application that corrects English grammar with a pre-trained T5 Transformer and teaches the learner why each change was made. The neural model handles the difficult part, producing a fluent correction, while a transparent rule-based engine splits the correction into individual edits, assigns each to one of 15 mistake categories, and explains it with a short lesson. A practice quiz built on real learner sentences and a progress tracker turn the tool from a corrector into a learning aid.

On the 747-sentence JFLEG test set, the model clearly outperformed the uncorrected baseline: GLEU rose from 0.787 to 0.911, and exact agreement with a human teacher more than doubled, from 26.6% to 61.3%. It improved 62.1% of sentences while making 7.4% worse. The project also showed that model accuracy is only part of a usable system: running on a real Windows laptop in China required handling proxy settings, a model mirror and duplicate downloads.

The main lessons learned were how encoder-decoder Transformers generate text, why evaluation needs a baseline and multiple references, how to turn model output into explainable feedback, and how to manage a project with Git branches, commits and a pull request.

6.2 Future Work

1. **Fairer evaluation:** test on a corpus the model has certainly not seen (for example BEA-2019 or CoNLL-2014) and compute the official JFLEG GLEU and the ERRANT F0.5 score.
2. **Better error labels:** replace the hand-written rules with ERRANT or a part-of-speech tagger (such as spaCy), so categories like verb tense and word form are detected from grammar rather than word lists.
3. **Compare or fine-tune models:** compare T5-base with larger models (for example FLAN-T5-large) or fine-tune on additional learner data, measuring the trade-off between accuracy and speed.
4. **Reduce over-correction:** show a confidence level per edit (from the beam-search scores) and let the user accept or reject each edit.
5. **Paragraph context:** correct several sentences together so that tense and pronoun consistency across sentences can be checked.
6. **Personalised practice:** choose quiz sentences that contain the learner's most frequent mistake types, and save progress between sessions with the user's consent.
7. **Deployment:** publish the app on Hugging Face Spaces so classmates can use it without installing Python.

References

1. Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. and Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv preprint arXiv:1906.02569*.
2. Bryant, C., Felice, M. and Briscoe, T. (2017). Automatic annotation and evaluation of error types for grammatical error correction. In *Proceedings of ACL 2017*, pp. 793–805.
3. Bryant, C., Felice, M., Andersen, Ø. E. and Briscoe, T. (2019). The BEA-2019 shared task on grammatical error correction. In *Proceedings of the 14th Workshop on Innovative Use of NLP for Building Educational Applications*, pp. 52–75.
4. Junczys-Dowmunt, M. and Grundkiewicz, R. (2016). Phrase-based machine translation is state-of-the-art for automatic grammatical error correction. In *Proceedings of EMNLP 2016*, pp. 1546–1556.
5. Junczys-Dowmunt, M., Grundkiewicz, R., Guha, S. and Heafield, K. (2018). Approaching neural grammatical error correction as a low-resource machine translation task. In *Proceedings of NAACL-HLT 2018*, pp. 595–606.
6. Lyster, R. and Ranta, L. (1997). Corrective feedback and learner uptake: Negotiation of form in communicative classrooms. *Studies in Second Language Acquisition*, 19(1), pp. 37–66.

7. Napoles, C., Sakaguchi, K., Post, M. and Tetreault, J. (2015). Ground truth for grammatical error correction metrics. In *Proceedings of ACL-IJCNLP 2015*, pp. 588–593.
8. Napoles, C., Sakaguchi, K. and Tetreault, J. (2017). JFLEG: A fluency corpus and benchmark for grammatical error correction. In *Proceedings of EACL 2017*, pp. 229–234.
9. Ng, H. T., Wu, S. M., Briscoe, T., Hadiwinoto, C., Susanto, R. H. and Bryant, C. (2014). The CoNLL-2014 shared task on grammatical error correction. In *Proceedings of CoNLL 2014: Shared Task*, pp. 1–14.
10. Omelianchuk, K., Atrasevych, V., Chernodub, A. and Skurzhanskyi, O. (2020). GECToR – Grammatical error correction: Tag, not rewrite. In *Proceedings of the 15th Workshop on Innovative Use of NLP for Building Educational Applications*, pp. 163–170.
11. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W. and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), pp. 1–67.
12. Rothe, S., Mallinson, J., Malmi, E., Krause, S. and Severyn, A. (2021). A simple recipe for multilingual grammatical error correction. In *Proceedings of ACL-IJCNLP 2021 (Short Papers)*, pp. 702–707.
13. Truscott, J. (1996). The case against grammar correction in L2 writing classes. *Language Learning*, 46(2), pp. 327–369.
14. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.
15. Wolf, T. et al. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations*, pp. 38–45.
16. Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M. et al. (2016). Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.
17. Yuan, Z. and Briscoe, T. (2016). Grammatical error correction using neural machine translation. In *Proceedings of NAACL-HLT 2016*, pp. 380–386.

Software, model and data used

- vennify. *t5-base-grammar-correction* (CC BY-NC-SA 4.0). huggingface.co/vennify/t5-base-grammar-correction
- JFLEG dataset (CC BY-NC-SA 4.0), loaded as `jhu-clsp/jfleg`. huggingface.co/datasets/jhu-clsp/jfleg
- Gradio gradio.app · Hugging Face Transformers huggingface.co/docs/transformers · PyTorch pytorch.org
- Project source code: folder `alh040` in github.com/xahiru/PT22026 (pull request #2)